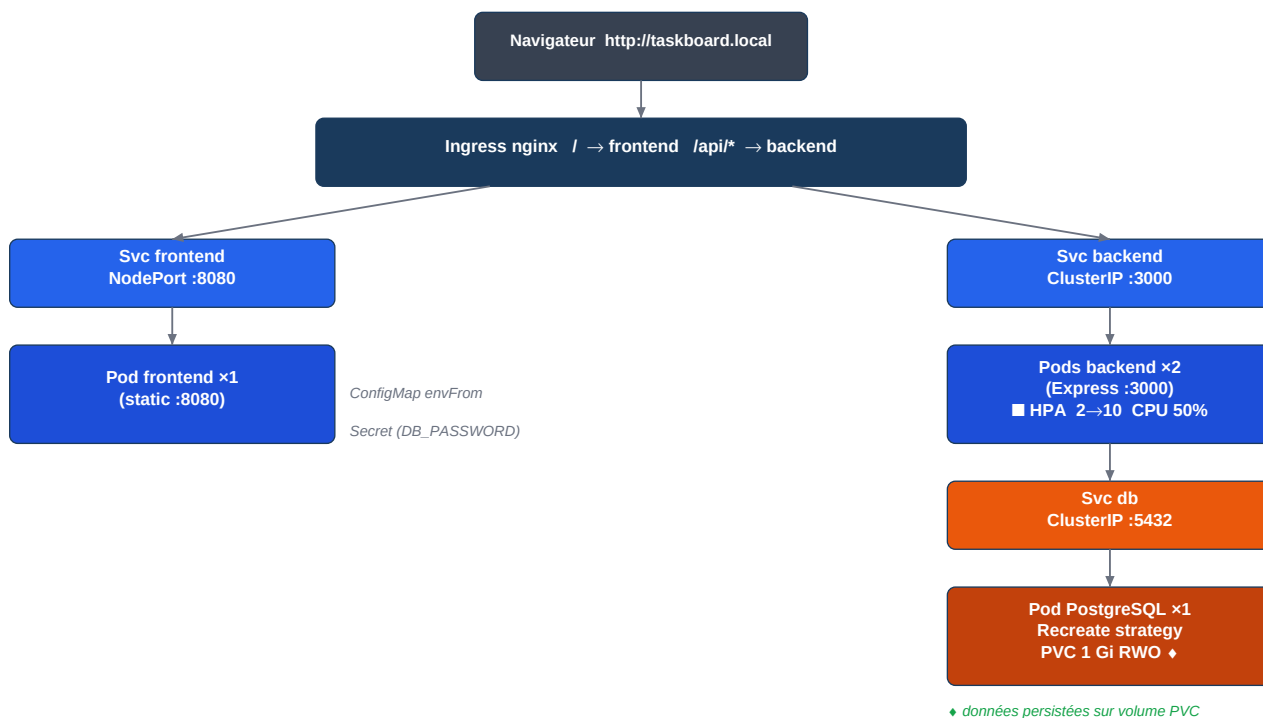


Rapport TP Kubernetes — TaskBoard

Josué Adami (Infrastructure) · Alexis Redaud (Robustesse) · Nicolas Gouy (Frontend & Exposition) — Cluster : Docker Desktop · Namespace : *taskboard*

1. Architecture Kubernetes



Objet	Ressource	Rôle
ConfigMap	taskboard-config / db-init	Config non sensible (DB_HOST, ports) + script init.sql
Secret	taskboard-secret	DB_PASSWORD en base64, jamais en clair dans un Deployment
PVC	taskboard-db-pvc	Stockage durable PostgreSQL — 1 Gi, ReadWriteOnce
Deployment	taskboard-backend	API Node.js — 2 replicas, startup/readiness/liveness probes, HPA
Deployment	taskboard-db	PostgreSQL 16-alpine — 1 replica, stratégie Recreate
Deployment	taskboard-frontend	UI statique — 1 replica, Service NodePort :8080
Ingress	taskboard-ingress	/ -> frontend, /api/* -> backend (rewrite)
HPA	taskboard-backend-hpa	Autoscaling backend : 2 -> 10 replicas, seuil CPU 50 %

2. Choix techniques justifiés

- **ConfigMap + Secret** — Toute la configuration non sensible vit dans *taskboard-config* (injectée via *envFrom*) ; le mot de passe DB est isolé dans *taskboard-secret* (base64). On change la config sans rebuild d'image et le secret n'apparaît jamais en clair dans un Deployment.
- **PVC pour PostgreSQL** — Le volume 1 Gi (ReadWriteOnce) est monté sur */var/lib/postgresql/data*. Sa durée de vie est découplée de celle du pod : les données survivent à toute suppression/recréation. La stratégie *Recreate* (vs *RollingUpdate*) évite deux writers simultanés sur le même volume.
- **Backend x2 replicas + probes** — Deux replicas assurent la haute disponibilité et servent de plancher au HPA (2→10, CPU 50 %). *startupProbe* /health (30×5 s) absorbe le démarrage lent de PostgreSQL ; *readinessProbe* /health retire le pod du Service si la DB est injoignable ; *livenessProbe* TCP:3000 vérifie Express **sans dépendre de la DB** (évite les *CrashLoopBackOff* en cascade).
- **DB en ClusterIP uniquement** — La base n'est jamais exposée hors du cluster. L'URL de l'API frontend est relative (*/api*) : l'Ingress route sur la même origine *taskboard.local*, sans injection de config côté client.

3. Preuves

```
PS C:\Users\nicol\ynov\DevOps\tp_taskboard> kubectl get all -n taskboard
NAME                                READY   STATUS    RESTARTS   AGE
pod/taskboard-backend-794dd86c7f-cv4vg  1/1     Running   1 (179s ago)  5h11m
pod/taskboard-backend-794dd86c7f-hdwf8  1/1     Running   1 (179s ago)  5h12m
pod/taskboard-db-56f4dc5b54-t8x92      1/1     Running   1 (179s ago)  5h39m
pod/taskboard-frontend-89f86c7c4-7fzrp  1/1     Running   1 (179s ago)  5h5m
```

```
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/taskboard-backend           ClusterIP           10.96.156.182   <none>            3800/TCP          5h39m
service/taskboard-db                ClusterIP           10.96.25.249   <none>            5432/TCP          5h39m
service/taskboard-frontend          NodePort           10.96.155.11   <none>            8080:38796/TCP    5h39m
```

```
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/taskboard-backend   2/2     2             2           5h39m
deployment.apps/taskboard-db        1/1     1             1           5h39m
deployment.apps/taskboard-frontend  1/1     1             1           5h39m
```

```
NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/taskboard-backend-565886c8b0  0         0         0       5h43m
replicaset.apps/taskboard-backend-794dd86c7f  2         2         2       5h11m
replicaset.apps/taskboard-backend-856c77fc9c  0         0         0       5h39m
replicaset.apps/taskboard-db-56f4dc5b54       1         1         1       5h39m
replicaset.apps/taskboard-frontend-869ff6d05  0         0         0       5h43m
replicaset.apps/taskboard-frontend-8c4f8ab658  0         0         0       5h39m
replicaset.apps/taskboard-frontend-78b66d869c  0         0         0       5h11m
replicaset.apps/taskboard-frontend-89f86c7c4  1         1         1       5h5m
```

```
NAME                                REFERENCE            TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/taskboard-backend-hpa  Deployment/taskboard-backend  CPU 13/58%      2         18         2           5h39m
```

① `kubectl get all -n taskboard` — stack complète Running

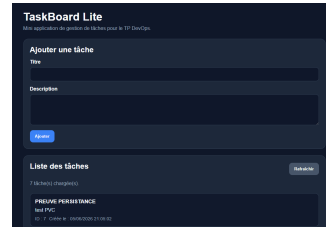
```
PS C:\Users\nicol\ynov\DevOps\tp_taskboard> kubectl delete pod -n taskboard -l component=database
pod "taskboard-db-56f4dc5b54-t8x92" deleted from taskboard namespace
PS C:\Users\nicol\ynov\DevOps\tp_taskboard> kubectl get pods -n taskboard -l component=database -w
NAME                                READY   STATUS    RESTARTS   AGE
taskboard-db-56f4dc5b54-qktvp      0/1     Running   0           6s
taskboard-db-56f4dc5b54-qktvp      1/1     Running   0          13s
```

③ Persistence (1/2) — pod PostgreSQL supprimé puis recréé

```
PS C:\Users\nicol\ynov\DevOps\tp_taskboard> kubectl get pods -n taskboard -l component=backend
NAME                                READY   STATUS    RESTARTS   AGE
taskboard-backend-794dd86c7f-cv4vg  1/1     Running   1 (3h1s ago)  5h12m
taskboard-backend-794dd86c7f-hdwf8  1/1     Running   1 (3h1s ago)  5h12m
```

```
PS C:\Users\nicol\ynov\DevOps\tp_taskboard> kubectl delete pod taskboard-backend-794dd86c7f-cv4vg -n taskboard
pod "taskboard-backend-794dd86c7f-cv4vg" deleted from taskboard namespace
PS C:\Users\nicol\ynov\DevOps\tp_taskboard> kubectl get pods -n taskboard -l component=backend
NAME                                READY   STATUS    RESTARTS   AGE
taskboard-backend-794dd86c7f-hdwf8  1/1     Running   1 (3h2m ago)  5h13m
taskboard-backend-794dd86c7f-mgglp  1/1     Running   0           58s
```

② Self-healing — pod supprimé, immédiatement recréé par le ReplicaSet



③ Persistence (2/2) — les tâches survivent grâce au PVC